End-to-End Data Pipeline for Food Delivery Industry on cloud (GCP)

ENGR - 516: Engineering Cloud Computing
Prof. Dingwen Tao

Team:
- Sakshi Gatyan (sgatyan@iu.edu)
- Bhargavi Jahagirdar (bjahagir@iu.edu)
- Arpita Lonakadi (arlona@iu.edu)

Video Presentation

https://youtu.be/AY60zrUDDgo

Introduction and Background

- Digitalization and online delivery surge generate vast data
- Crucial for predictive insights, strategic franchising, and operational optimization
- Emphasizes data-driven decision-making for growth
- Challenges in Data Utilization
    - Data volume and standardization inconsistencies hinder efficiency
    - Lack of standardized protocols and immature data collection
    - Cloud computing face scalability, security, and traceability issues

Motivation

Providing actionable insights to the food delivery system for data-driven decision making by resolving data challenges

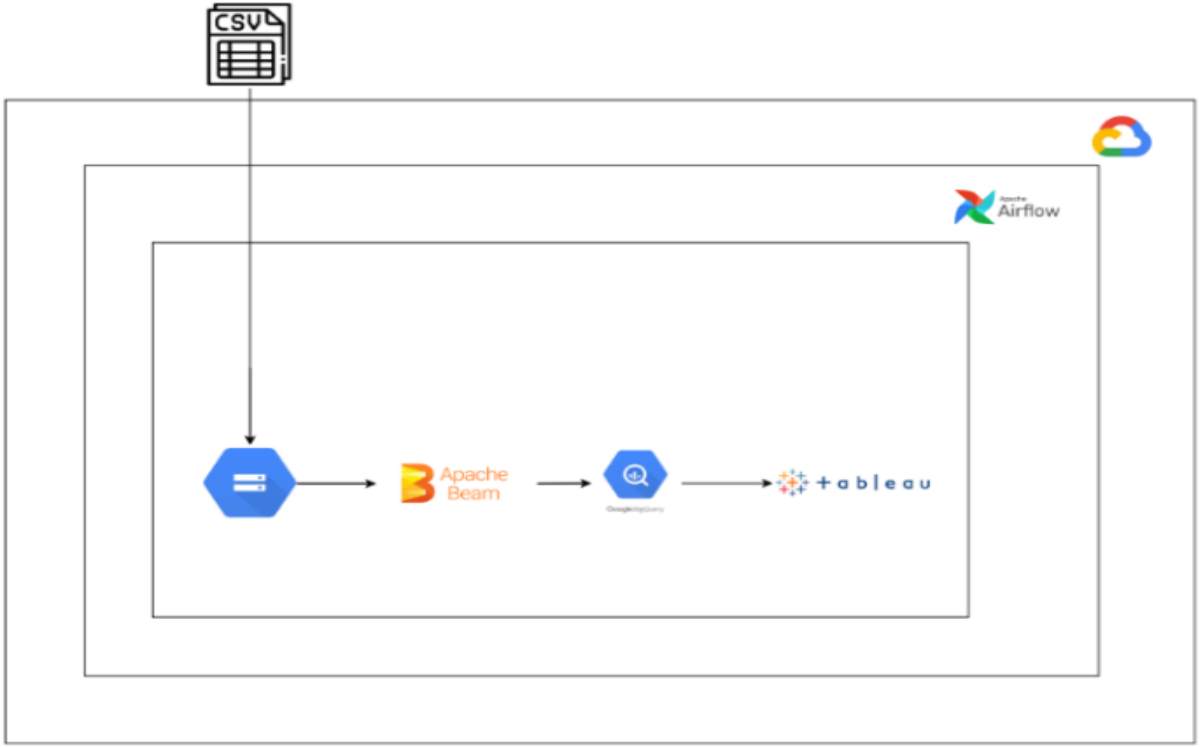
Technological Solutions for Data Challenges

- Cloud computing and enhanced processing techniques offer scalable solutions
- Integrating traditional and innovative methods optimizes data processing
- Leveraging Python ETL, Apache Airflow, and Apache Beam for parallel processing
- Promises to bridge gaps and enhance scalability in the food industry

Dataset creation

- Found a dataset from Kaggle: link
- Augmented the dataset to simulate a real-life Big Data generation system
- Fields:
  - Customer_id
  - date
  - time
  - order_id
  - items
  - amount
  - mode
  - restaurant
  - Status
  - ratings
  - feedback
  - delivery_time
  - order_preparation_time
  - Premium_member
  - restaurant_location

Project Architecture & Design



Resource Creation and Workflow

Description of architecture diagram:

1. Google Cloud Storage (GCS) is utilised for storing CSV files, providing a reliable and scalable data storage solution

2. Apache Airflow is employed for orchestrating ETL jobs, ensuring efficient and monitored workflow execution

3. Apache Beam is tasked with data processing, transformation, and loading processed data into Google BigQuery

4. Google BigQuery is a data warehousing solution known for its scalability and real-time analytics capabilities.

5. Tableau serves as our BI tool, leveraging data from Google BigQuery to generate insightful reports and visualisations.

Apache Beam Data Pipeline

Apache Beam is an open-source unified programming model designed for both batch and stream processing of large-scale data sets. It provides a portable and expressive API for defining data processing pipelines that can run on various distributed processing backends, including Apache Spark, Apache Flink, Google Cloud Dataflow, and others

Following are the steps followed in the apache-beam code:

1. **Setup and Imports:** Import necessary libraries and setup command-line argument parsing.
2. **Pipeline Initialization:** Define PipelineOptions and create a Beam pipeline object.
3. **Transformation Functions:** Define functions to process data.
4. **Data Processing Pipeline**: Read data, apply transformations, and split data as needed.
5. **Counting Operations**: Calculate metrics on the data.
6. **BigQuery Dataset Creation:** Ensure dataset existence or create if needed.
7. **Define BigQuery Schema:** Define schema for target tables.
8. **Write to BigQuery**: Convert data and write to BigQuery tables.
9. **Running the Pipeline:** Execute the pipeline and handle errors if any.

Apache Beam ETL data pipeline on GCP (Logic)

Various transformations which we used in the pipeline:

Data Cleaning: Cleaning the data using by removing special characters, colons and deduplicating the records ingested from the Storage bucket using beam FILTER and DISTINCT beam functions

Data Transformation: We performed normalization on certain columns like rating to ensure it is in a standard range. Additionally, we performed standardization on some columns like date with datetime data type. Aggregation was performed using GroupByKey, Combine, ParDo functions to get the summarized information.

Data Validation and Error Handling: We performed data validation and error handling which involved verifying the integrity, accuracy, and consistency of the final dataset that will be loaded into the BQ data warehouse. By validating the data against predefined rules and handling errors effectively, reliability of the analyses and decision-making processes can be improved.

C: > Users > Bhargavi Jahagirdar > OneDrive > Desktop > Books > Semester 2 > Engineering Cloud Computing > Project > beam.py > ...

```python
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
import argparse
from google.cloud import bigquery
from datetime import datetime, time

# Command-line argument parser
parser = argparse.ArgumentParser()
parser.add_argument('--input', dest='input', required=True, help='Input file to process.')
path_args, pipeline_args = parser.parse_known_args()
inputs_pattern = path_args.input

# Pipeline options
options = PipelineOptions(pipeline_args)
p = beam.Pipeline(options=options)

# Transformation functions
def remove_last_colon(row):
    # Remove the trailing colon from the fifth column of the row
    cols = row.split(',')
    item = str(cols[4])
    if item.endswith(':'):
        cols[4] = item[:-1]
    return ','.join(cols)

def remove_special_characters(row):
    # Remove special characters from each column of the row
    import re
    cols = row.split(',')
    ret = ''
    for col in cols:
        clean_col = re.sub(r'[?%&]', '', col)
        ret = ret + clean_col + ','
    ret = ret[:-1]
    return ret

```

C: > Users > Bhargavi Jahagirdar > OneDrive > Desktop > Books > Semester 2 > Engineering Cloud Computing > Project > beam.py > ...

```python
cleaned_data = (
    p
    | beam.io.ReadFromText(inputs_pattern, skip_header_lines=1)
    | beam.Map(remove_last_colon)
    | beam.Map(lambda row: row.lower())
    | beam.Map(remove_special_characters)
    | beam.Map(lambda row: row + ',1')
)

delivered_orders = cleaned_data | 'delivered filter' >> beam.Filter(lambda row: row.split(',')[8].lower() == 'delivered')
other_orders = cleaned_data | 'Undelivered Filter' >> beam.Filter(lambda row: row.split(',')[8].lower() != 'delivered')

# Counting operations
total_count = (
    cleaned_data
    | 'count total' >> beam.combiners.Count.Globally()
    | 'total map' >> beam.Map(lambda x: 'Total Count:' + str(x))
    | 'print total' >> beam.Map(print_row)
)

delivered_count = (
    delivered_orders
    | 'count delivered' >> beam.combiners.Count.Globally()
    | 'delivered map' >> beam.Map(lambda x: 'Delivered count:' + str(x))
    | 'print delivered count' >> beam.Map(print_row)
)

undelivered_count = (
    other_orders
    | 'count others' >> beam.combiners.Count.Globally()
    | 'other map' >> beam.Map(lambda x: 'Others count:' + str(x))
    | 'print undelivered' >> beam.Map(print_row)
)

# BigQuery dataset creation
client = bigquery.Client()
dataset_id = "ecc-food-delivery-dev-421017.food_orders_dataset"
```

Airflow Orchestration

Apache Airflow is an open-source platform for orchestrating workflows in data pipeline. It allows to automate tasks that involve moving data around, transforming it, and running different programs or models.

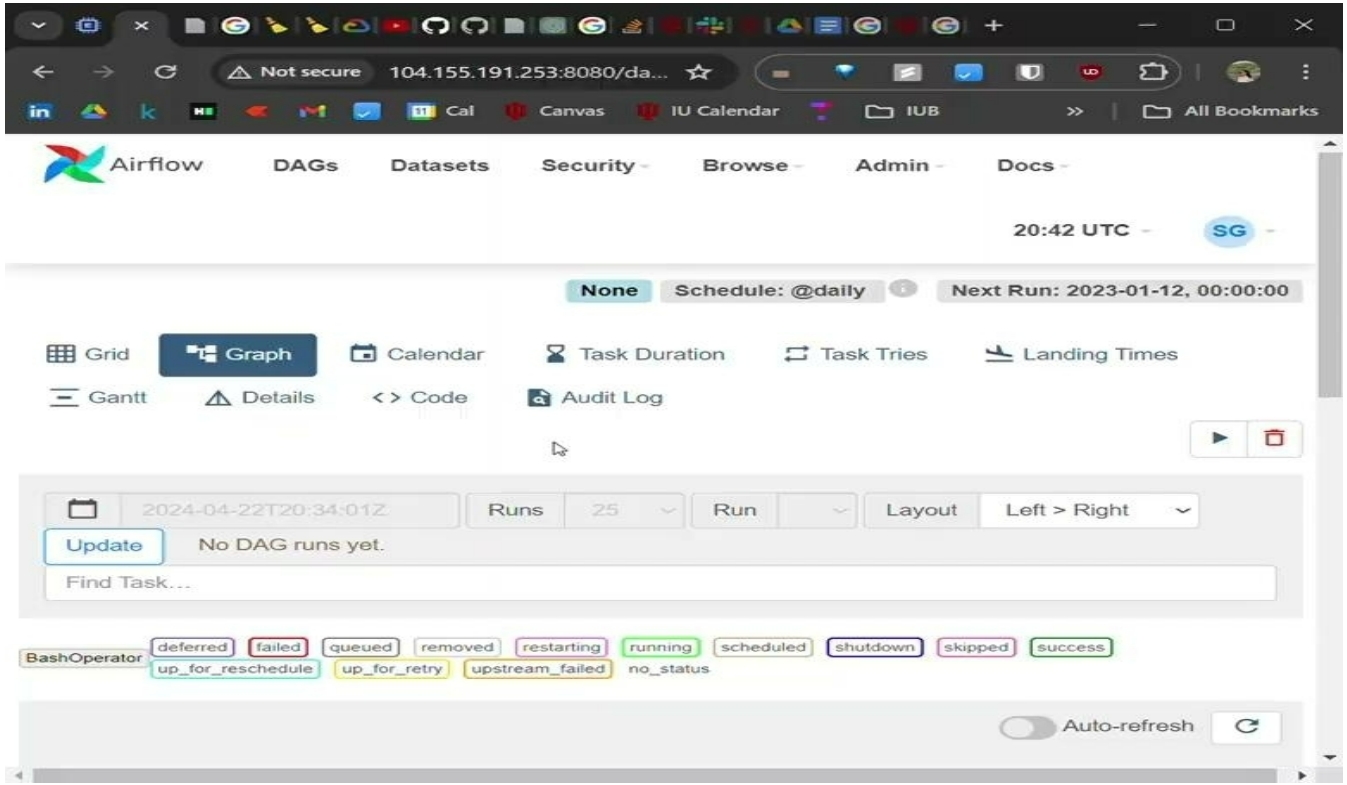
We hosted Airflow version 2.5.0 on GCP VM instance, with the use of airflow our aim was to automate data pipeline which could sense the presence of data files in GCS bucket and run the beam transformation pipeline and finally update the data into Bigquery warehouse from where it can be used for further analysis.

We created a python DAG file which has the following tasks:

1. **GCS Sensor**: monitors the presence of files with a specific prefix “food_daily” in the GCS bucket and trigger downstream tasks based on the existence or absence of these files
2. **List Files**: manages files in a Google Cloud Storage (GCS) bucket. It passes the bucket name and prefix as arguments to the function and pushes the return value to XCom for further processing by downstream apache-beam ETL pipeline in the Airflow DAG.
3. **Beam Task**: executing a Beam pipeline defined in the 'beam.py' file stored in GCS

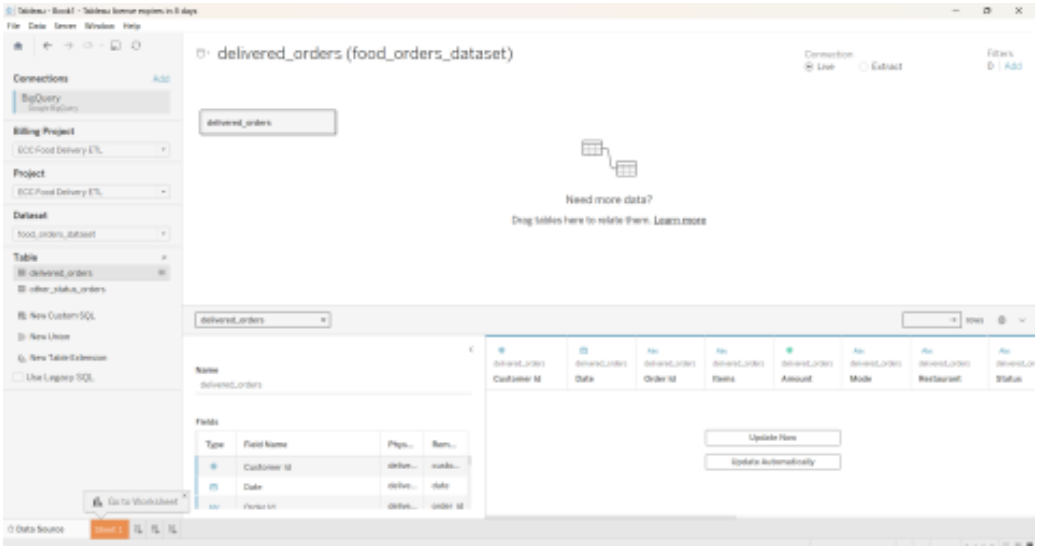
Link to demo recording: https://drive.google.com/file/d/1aoftZ9PcnZn0G7ICA6nUDD9PI3EFcgLs/view?usp=drive_link

Airflow Dag orchestration Run on GCP VM instance



BigQuery to Tableau

Connected Tableau to the BigQuery dataset using OAuth



Results

On analysing the data obtained we have found the following observations. The observations are grouped under 3 types i.e :

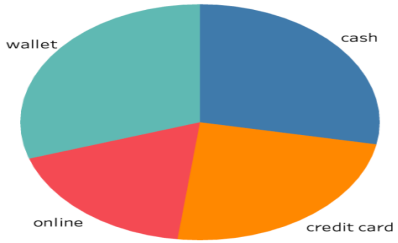
Overall metrics : Useful to get an overall understanding of best restaurants, top customers, order purchase method frequency, restaurant rating distribution .

Restaurant statistic metrics : Useful to strategically analyse how restaurants are doing across different locations and how are their order frequencies. Helps in taking franchising decisions

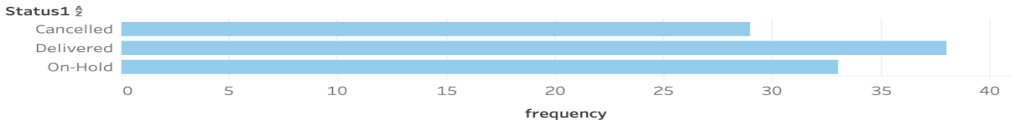
Customer statistic metrics: Useful to analyse the population of customers spread across geographies and how customers place orders across locations. Helps to understand landscape and customer behaviour across locations

Overall Metrics

Delivered order distribuion

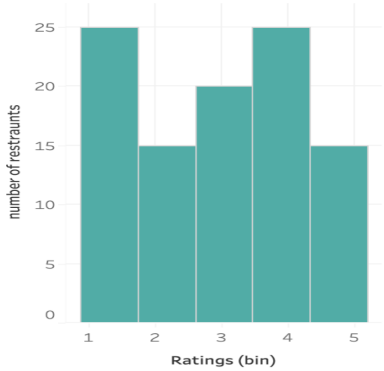


Distribution of all orders



Top 5 food items

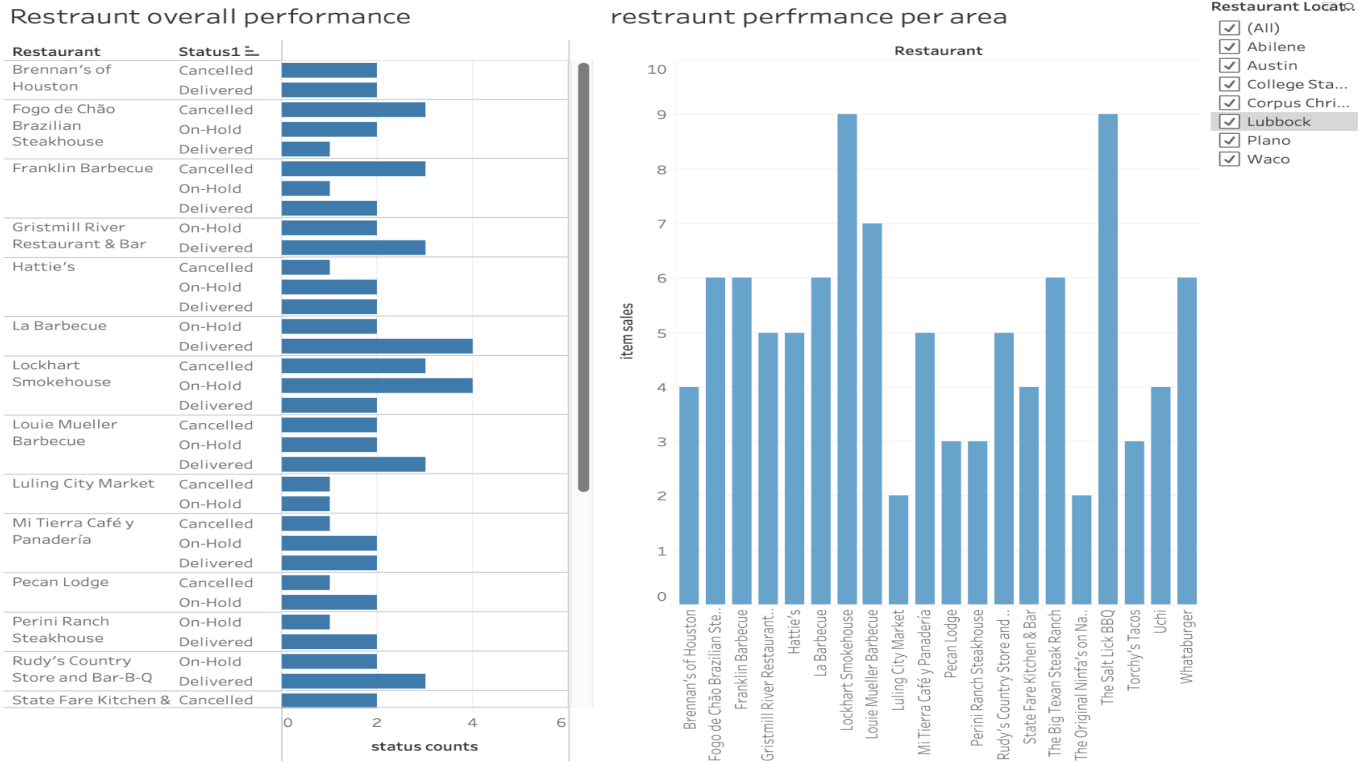
Restraunt rating distribution



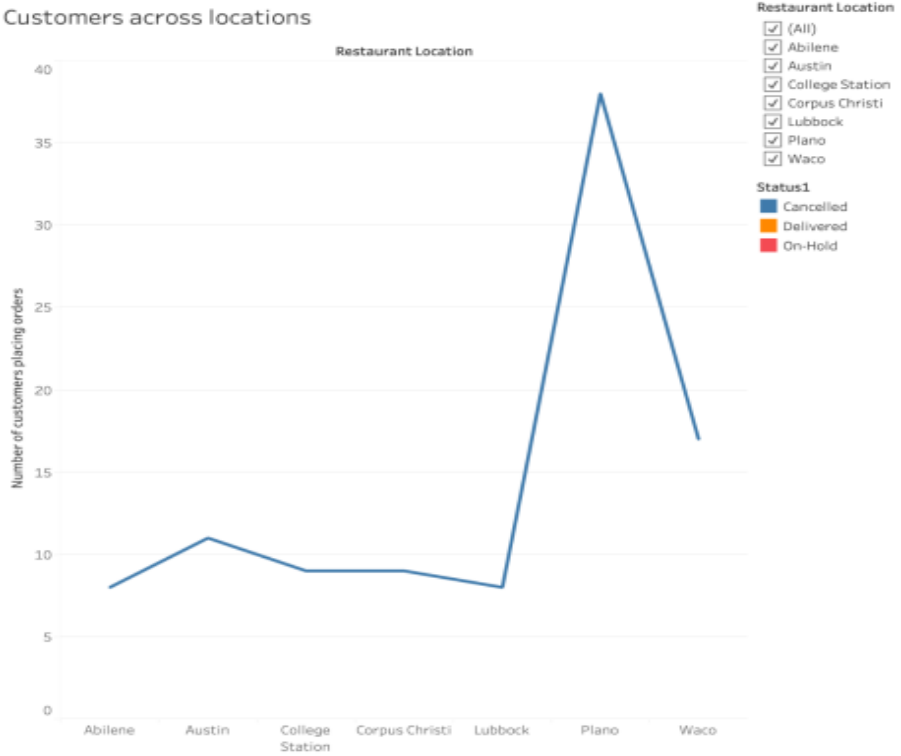
Top 5 restaurants

Popular customers distribution

Restaurant statistics



Customer Statistics



Future Work Discussion

On using cloud, we were effectively able to scale the project and draw valuable insights which could help in decision making.

One of the key components that our data covers is distribution of restaurants and franchise chains across locations. Our insights greatly help in analysing the consumption diaspora and how one can strategically open new chains in places where demand is visible. We also capture the overall performance of restaurants and payment modes.

- Many more valuable insights like categorizing restaurant chains or generating order frequency distribution based on category can help competitors analyse the diaspora better.
- Analysing and correlating order status with restaurant ratings can give insights on customer prioritization.
- Payment mode distributions across regions can help automate and move food business platforms online in a beneficial way.

With the following future works which require more levels of data capture and analytics, one can more significantly impact decisions for business growth and also scalability. Using cloud computing can help greatly to create visible impact in the analytics given the sheer volume of data and established use methodology as demonstrated in our project.